

Lucas Augusto da Silva Cardoso (202200332300)

DCA3703 | Programação Paralela | Relato de experimento

TAREFA 13: AVALIAÇÃO DA ESCALABILIDADE COM AFINIDADE DE THREADS

1. EXPERIMENTO

Avalie como a escalabilidade do seu código de Navier-Stokes muda ao utilizar os diversos tipos de afinidades de threads suportados pelo sistema operacional e pelo OpenMP no mesmo nó de computação do NPAD que utilizou para a tarefa 12.

2. CONCEITOS RELEVANTES

- **Afinidade de threads:** define onde as threads podem executar. A fixação pode reduzir migrações entre núcleos e favorecer o reaproveitamento da memória cache.
- **`OMP_PLACES`:** define os locais disponíveis para execução. Neste experimento, `cores` utiliza os núcleos físicos como locais.
- **`OMP_PROC_BIND`:** controla a distribuição das threads nesses locais. `false` desabilita a fixação pelo OpenMP; `true` habilita uma política definida pela implementação; `close` aproxima as threads; `spread` as espalha; e `master` concentra todas no local da thread principal.
- **Localidade de memória e NUMA:** o custo de acesso à memória pode variar conforme a localização dos dados e da thread. A inicialização paralela pode favorecer essa proximidade, mas o posicionamento das páginas precisa ser medido para confirmar esse efeito.

3. MÁQUINA UTILIZADA

Os experimentos foram submetidos ao NPAD/UFRN, na partição amd-512, solicitando um nó exclusivo, 32 CPUs e uma thread por núcleo físico. Foram avaliadas configurações com 1, 4, 8, 16 e 32 threads, utilizando OMP_PLACES=cores.

4. PROCEDIMENTO EXPERIMENTAL

A mesma implementação foi executada com as cinco políticas de afinidade, mantendo as opções de compilação e “OMP_PLACES=cores”. Foram utilizadas 1, 4, 8, 16 e 32 threads, com 300 passos e cinco repetições por configuração.

As malhas internas de 256 × 256 a 2.048 × 2.048 pontos permitem avaliar o aumento das threads com uma entrada fixa. Também foi preparada uma sequência com aproximadamente 65.536 pontos por thread para avaliar a escalabilidade fraca, registrando as diferenças decorrentes do arredondamento das dimensões.

Cada execução realiza um aquecimento antes das medições. Os tempos de reinicialização e evolução são registrados separadamente em CSV. Uma bateria adicional utiliza o PaScal Analyzer para gerar um JSON por política. O posicionamento das threads é inspecionado por um programa auxiliar, fora da medição do solver.

O mesmo nó utilizado na tarefa 12 foi utilizado, conforme orientado.

5. CÓDIGO

Foi utilizada a terceira versão do código da tarefa 12 (V3), que mantém uma região paralela durante os passos da simulação, utiliza ponteiros privados para alternar os buffers e aplica “omp simd” ao laço interno. A inicialização da malha também é paralela.

Figura 3: Versão do código utilizado

```
#include "navierStokes.h"

void evoluir(Grade *g, int passos) {
  int n=g->n; double inv=(double)(n-1)*(n-1), fator=0.2/inv;
  double *u=g->u,*v=g->v,*un=g->un,*vn=g->vn;
  #pragma omp parallel firstprivate(u,v,un,vn)
  {
```

```
    for (int p=0;p<passos;p++) {
      #pragma omp for schedule(static)
      for (int i=1;i<n-1;i++) {
        #pragma omp simd
        for (int j=1;j<n-1;j++) {
          ATUALIZAR;
        }
      }
      /* Barreira do omp for termina todas as escritas antes da troca.
        Ponteiros privados dispensam single e segunda barreira. */
      double *t=u;u=un;un=t;t=v;v=vn;vn=t;
    }
  }
  if (passos%2) { double *t=u;u=un;un=t;t=v;v=vn;vn=t; }
  g->u=u;g->v=v;g->un=un;g->vn=vn;
}
```

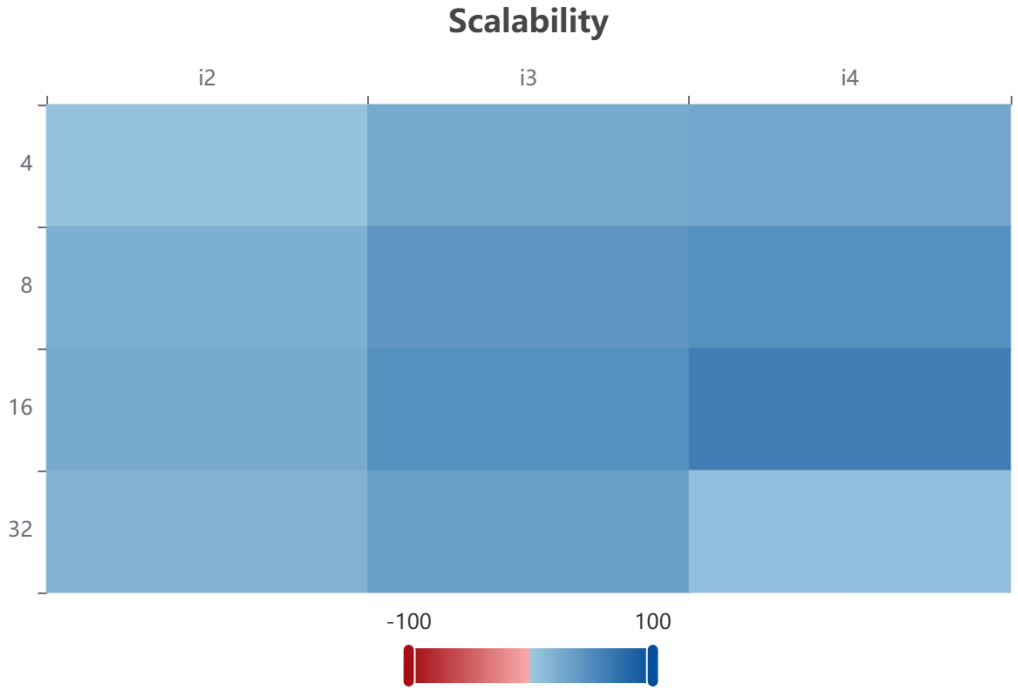
6. RESULTADOS

Foram comparadas cinco políticas de afinidade, mantendo a implementação V3 e variando a quantidade de threads e o tamanho da malha. As entradas i1 a i4 correspondem a 65.536, 262.144, 1.048.576 e 4.194.304 pontos internos, respectivamente. Cada entrada quadruplica o trabalho por passo.

Nos mapas abaixo, azul mais intenso indica resultado mais favorável no índice de escalabilidade do Viewer; vermelho indica índice negativo.

5.1. false

Figura 4: Afinidade false



O azul se intensifica até 16 threads nas maiores entradas, mas enfraquece em i4 com 32 threads. Os tempos confirmam essa perda. Migrações e competição por recursos podem contribuir, mas não foram medidas.

5.2. true e close

Figura 5: Afinidade true

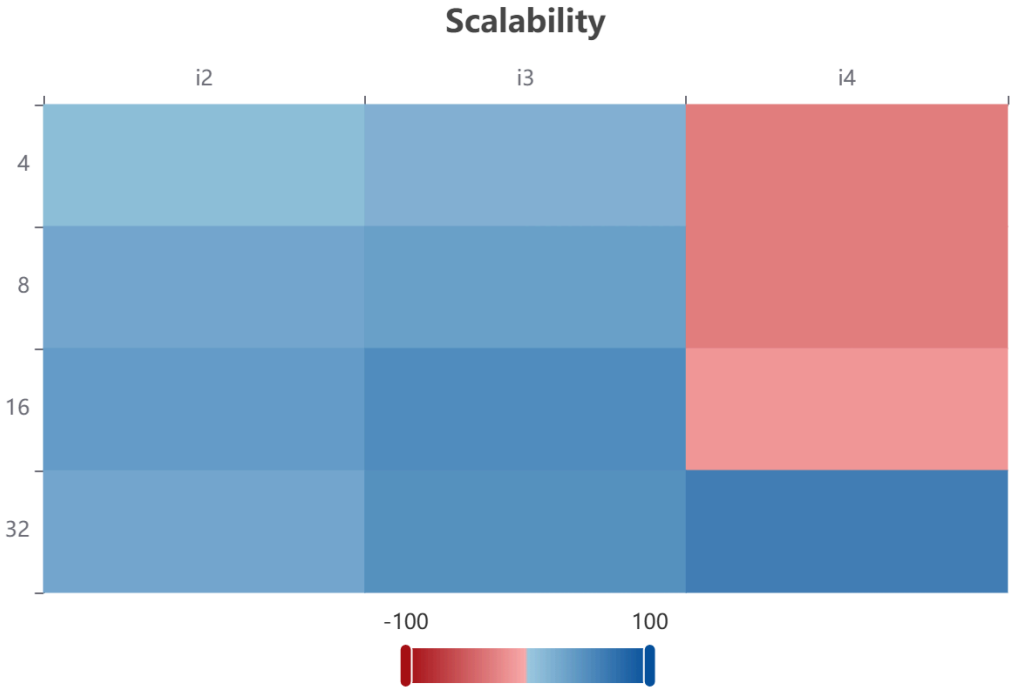
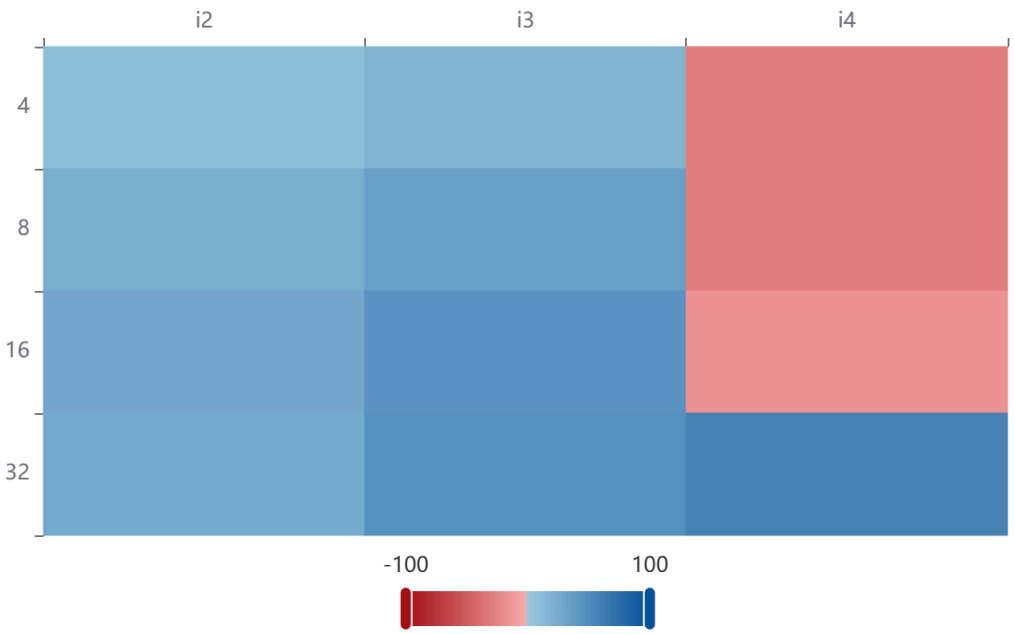


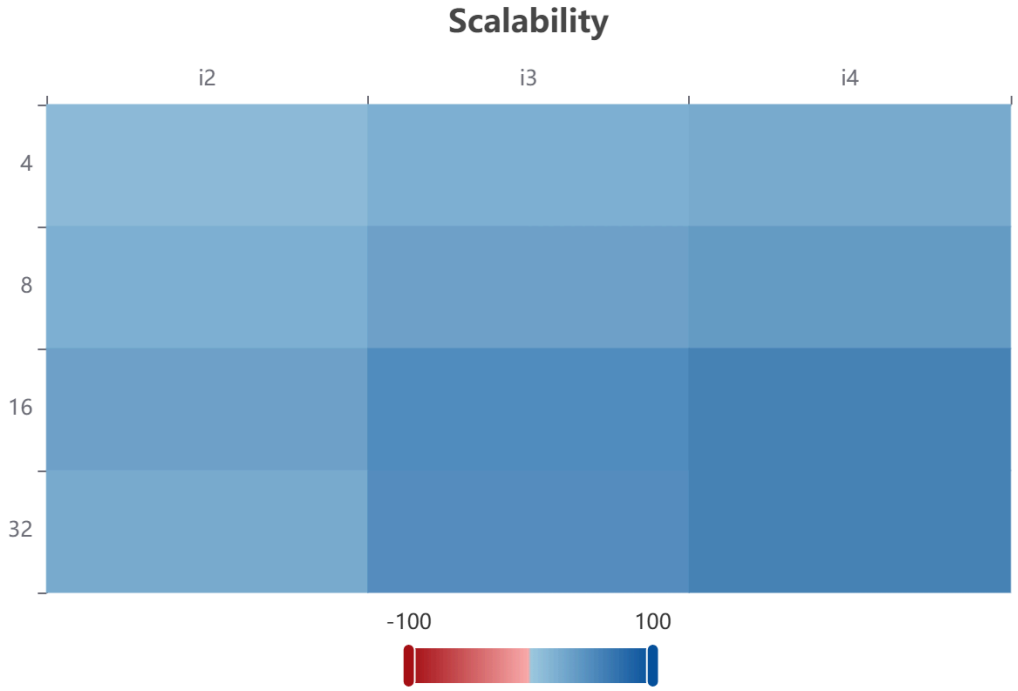
Figura 6: Afinidade close



Apresentam gráficos semelhantes, coerentemente com as mesmas máscaras de afinidade registradas. A faixa vermelha em i4 com 4 a 16 threads indica dificuldade de aproveitar o crescimento da entrada. A concentração em núcleos próximos pode contribuir; com 32 threads, o resultado melhora.

5.2. spread

Figura 7: Afinidade spread



Mantém o azul e ganha intensidade nas maiores entradas. A distribuição das threads pelos núcleos disponíveis pode favorecer o uso de recursos de cache e memória. Foi a política com redução mais consistente do tempo na maior malha.

7. CONSIDERAÇÕES FINAIS

O experimento mostrou que a afinidade das threads influencia o desempenho e a escalabilidade mesmo sem alterar o cálculo do programa. A política spread apresentou redução mais consistente do tempo na maior entrada, enquanto master evidenciou o prejuízo de concentrar muitas threads em um único núcleo. Com 32 threads, true, close e spread ocuparam os mesmos lugares e apresentaram tempos próximos.

Portanto, aumentar a quantidade de threads exige considerar também sua distribuição e o tamanho do problema. A melhor política depende dos recursos efetivamente disponíveis; possíveis efeitos de cache e NUMA ajudam a interpretar os resultados, mas não foram medidos diretamente.

8. DEMAIS CÓDIGOS

Figura x: navierStokes.h

```
#ifndef NAVIER_STOKES_H
#define NAVIER_STOKES_H
#include <stddef.h>

typedef struct { int n; double *u, *v, *un, *vn; } Grade;
void evoluir(Grade *g, int passos);

/* Mesma ordem aritmetica do solver original. CFL: sigma_x+sigma_y=0.4. */
#define ATUALIZAR \
  size_t k=(size_t)i*n+j; \
  un[k]=u[k]+fator*((u[k+n]-2*u[k]+u[k-n])*inv+(u[k+1]-2*u[k]+u[k-1])*inv); \
  vn[k]=v[k]+fator*((v[k+n]-2*v[k]+v[k-n])*inv+(v[k+1]-2*v[k]+v[k-1])*inv)
#endif
```

Figura 15: navierStokes.c

```
#include <errno.h>
#include <limits.h>
#include <math.h>
#include <omp.h>
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#ifdef USAR_PASCAL
#include "pascalops.h"
#define INICIAR_REGIAO(id) pascal_start(id)
#define TERMINAR_REGIAO(id) pascal_stop(id)
#else
#define INICIAR_REGIAO(id) ((void)0)
#define TERMINAR_REGIAO(id) ((void)0)
#endif

#include "navierStokes.h"
#ifndef VERSAO
#error Defina VERSAO ao compilar com uma implementacao navierStokesV1, V2 ou V3.
#endif

static int inteiro(const char *s, int minimo, int maximo) {
  char *fim; errno = 0; long x = strtol(s, &fim, 10);
```

```
    if (errno || !*s || *fim || x < minimo || x > maximo) {
        fprintf(stderr, "Argumento invalido: %s\n", s); exit(EXIT_FAILURE);
    }
    return (int)x;
}

static Grade criar(int n) {
    Grade g = {.n=n}; size_t bytes = (size_t)n*n*sizeof(double);
    g.u=malloc(bytes); g.v=malloc(bytes); g.un=malloc(bytes); g.vn=malloc(bytes);
    if (!g.u || !g.v || !g.un || !g.vn) { fputs("Falha de alocacao\n",stderr); exit(1); }
    return g;
}

static void liberarMemoria(Grade g) {
    free(g.u);
    free(g.v);
    free(g.un);
    free(g.vn);
}

static void inicializar(Grade g, int paralelo, int cenario) {
    int n=g.n;

    #pragma omp parallel for if(paralelo) schedule(static)
    for (int i=0;i<n;i++) for (int j=0;j<n;j++) {
        size_t k=(size_t)i*n+j;

        double x=(double)i/(n-1)-0.5, y=(double)j/(n-1)-0.5;
        double u=cenario==0 ? 0 : cenario==1 ? 1.5 : 2*exp(-(x*x+y*y)/(2*0.08*0.08));
        double v=cenario==1 ? 0.5 : u*0.5;

        if (cenario!=1 && (i==0 || j==0 || i==n-1 || j==n-1)) u=v=0;

        g.u[k]=g.un[k]=u; g.v[k]=g.vn[k]=v;
    }
}
static void referencia(Grade *g, int passos) {
    int n=g->n; double inv=(double)(n-1)*(n-1), fator=0.2/inv;
    double *u=g->u,*v=g->v,*un=g->un,*vn=g->vn;
    for (int p=0;p<passos;p++) {
        for (int i=1;i<n-1;i++) for (int j=1;j<n-1;j++) { ATUALIZAR; }
        double *t=u;u=un;un=t;t=v;v=vn;vn=t;
    }
```

```
    g->u=u;g->v=v;g->un=un;g->vn=vn;
}

static int validar(void) {
    int falhas=0;
    for (int c=0;c<3;c++) for (int passos=1;passos<=12;passos+=11) {
        Grade ref=criar(31);omp_set_num_threads(1);inicializar(ref,0,c);referencia(&ref,passos);
        for (int th=1;th<=4;th*=2) {
                                                                                Grade
g=criar(31);omp_set_num_threads(th);inicializar(g,VERSAO==3,c);evoluir(&g,passos);
            for (size_t k=0;k<31*31;k++) {
                                    if  (!isfinite(g.u[k])  ||  !isfinite(g.v[k])  ||  fabs(g.u[k]-ref.u[k])>1e-12  ||
fabs(g.v[k]-ref.v[k])>1e-12) falhas++;
                if (c<2 && (g.u[k]!=(c==1?1.5:0) || g.v[k]!=(c==1?0.5:0))) falhas++;
                if (c==2 && (g.u[k]<0 || g.u[k]>2 || fabs(g.v[k]-g.u[k]*0.5)>1e-12)) falhas++;
            }
            liberarMemoria(g);
        }
        liberarMemoria(ref);
    }
      printf("Validacao  V%d:  %s  (%d  falhas)\n",VERSAO,falhas?"FALHOU":"OK",falhas);return
falhas?1:0;
}

int main(int argc,char **argv) {
    omp_set_dynamic(0);
    if (argc==2 && strcmp(argv[1], "--validar")==0) return validar();
#ifdef USAR_PASCAL
    if (argc!=3) {
                            fprintf(stderr,"Uso:   %s   PONTOS_INTERNOS   PASSOS;   threads   via
OMP_NUM_THREADS\n",argv[0]);return 1;
    }
    int pontos=inteiro(argv[1],36,268369924);
    int lado=(int)sqrt((double)pontos);
    if ((long long)lado*lado!=pontos) {
        fputs("PONTOS_INTERNOS deve ser um quadrado perfeito\n",stderr);return 1;
    }
    int ver=VERSAO,n=lado+2,passos=inteiro(argv[2],1,1000000),th=omp_get_max_threads();
#else
                if   (argc!=4)   {   fprintf(stderr,"Uso:   %s   N(8..16384)   PASSOS   THREADS   |
--validar\n",argv[0]);return 1; }
```

```
                                                                                int
ver=VERSAO,n=inteiro(argv[1],8,16384),passos=inteiro(argv[2],1,1000000),th=inteiro(argv[3],
1,4096);
#endif
  omp_set_num_threads(th);int reais=0;
  #pragma omp parallel
  {
    #pragma omp single
    reais=omp_get_num_threads();
  }

  if (reais!=th) {
    fprintf(stderr,"Solicitadas %d threads, obtidas %d\n",th,reais);return 1;
  }

  Grade g=criar(n);inicializar(g,ver==3,2);evoluir(&g,5); /* aquecimento */

  INICIAR_REGIAO(1);
  inicializar(g,ver==3,2);
  TERMINAR_REGIAO(1);

  INICIAR_REGIAO(2);
                                double        inicio=omp_get_wtime();evoluir(&g,passos);double
tempo=omp_get_wtime()-inicio;
  TERMINAR_REGIAO(2);

  double soma=0,pico=0;
  for (size_t k=0;k<(size_t)n*n;k++) {
    soma+=g.u[k]+g.v[k];if(g.u[k]>pico)pico=g.u[k];
  }
  if (!isfinite(soma) || pico>2 || pico<0) {
    fputs("Resultado numerico invalido\n",stderr);
    liberarMemoria(g);
    return 1;
  }

  printf(
    "%d,%d,%d,%d,%.9f,%.17g,%.17g\n",
    ver,
    n,
    passos,
    reais,
    tempo,
```

```
    soma,
    pico
  );

  liberarMemoria(g);return 0;
}
```